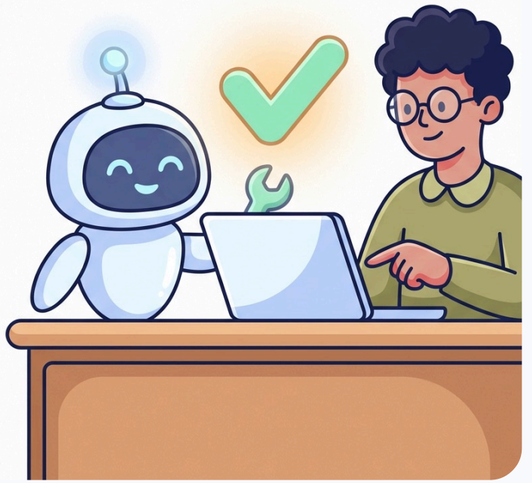
Лекция · агентская разработка

Как бежать быстро и не сломаться

— Николай Иготти

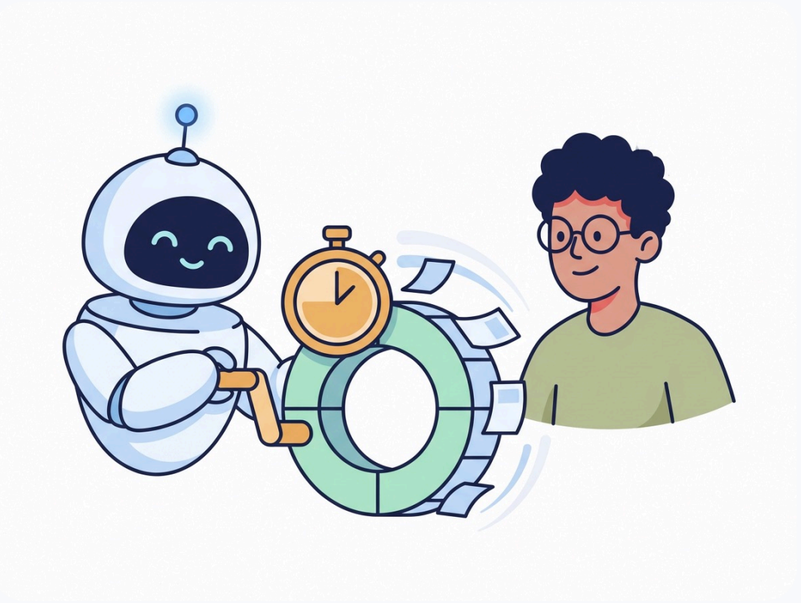
Цикл «написал → проверил» как единица работы



Что это такое

Агент — это цикл, не автодополнение

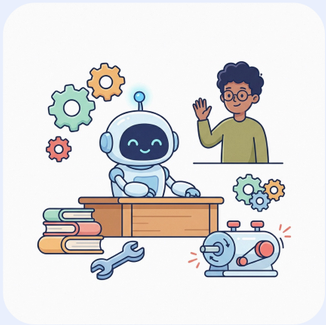
- Пишет: сотни строк кода за секунды
- Запускает: компилирует и выполняет сам
- Проверяет: видит результат, а не код
- Повторяет: правит и запускает снова



Организующая рамка

Три составляющих быстрой разработки

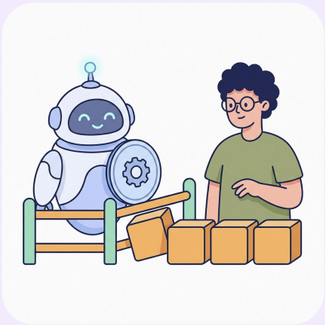
Инструменты



язык, компилятор, среда

дают скорость

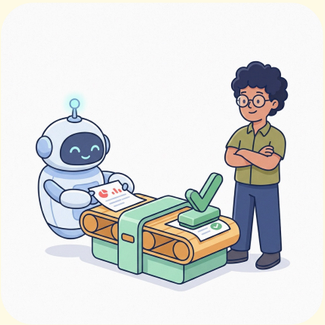
Принципы



стандарты и соглашения

дают стабильность

Процессы



документация, знания

дают темп

Почему это проблема

Три трудности, которые создаёт агент

Дорогая проверка

проверка не дешевле генерации

значит цикл проверки — короче

Вероятностные ошибки

каждая строка может быть неверной

ловить на самом раннем слое

Забывание

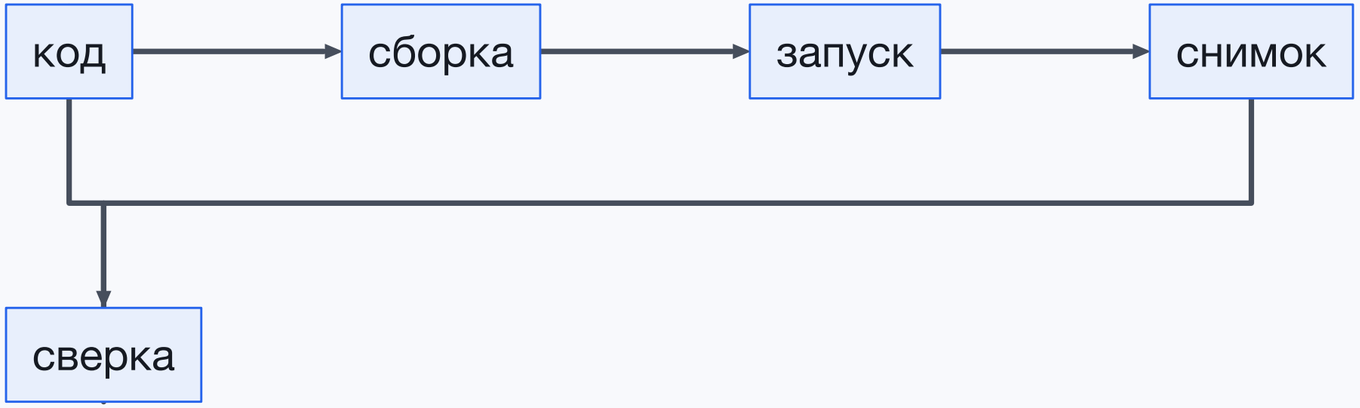
контекст короток

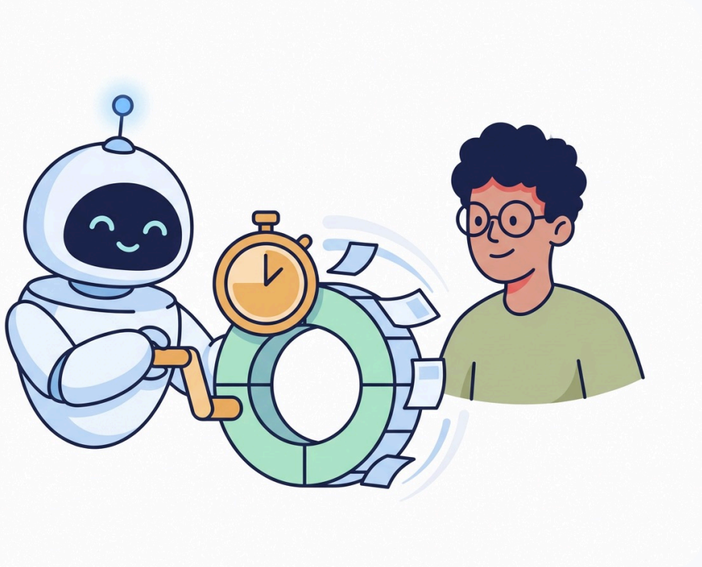
знания — во внешней памяти

Сердце агентной разработки

Главный цикл: секунды, не минуты

И снова по кругу: один оборот цикла и есть единица работы агента

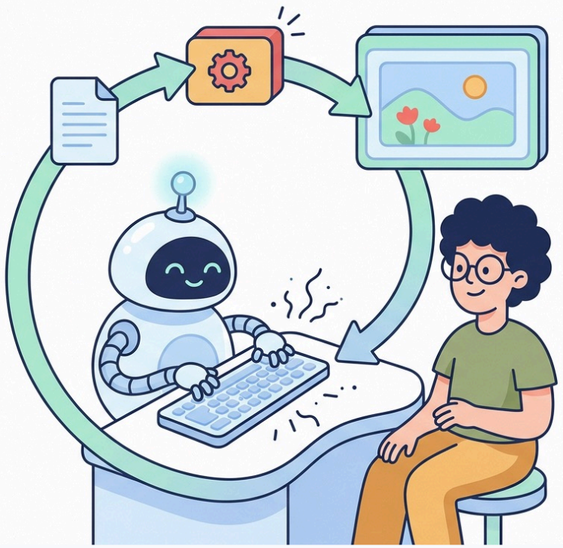




Цикл по цифрам

Полный круг за секунды

- Генерация кода — 1–5 с (инференс)
- Компиляция — 0,5–2 с (типы)
- Запуск — 0,2 с (среда)
- Проверка — 0,5 с (рендер)
- Полный цикл — ≈2–8 с



Вопрос аудитории

Что труднее: код или задача?

A Написать код

быстро выдать рабочую версию

модель видела миллионы примеров

ошибку видно сразу: код не компилируется

B Понять, что именно нужно

уточнить требования и границы

решить, что считается правильным

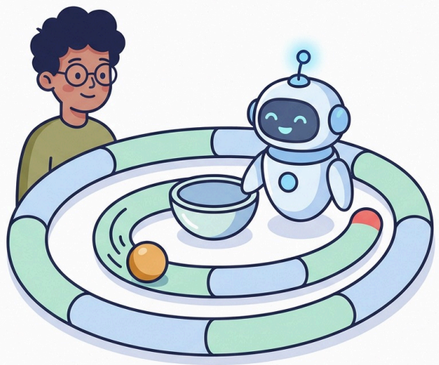
ошибку не видно: код работает, но не то

Правильный ответ — понять задачу: требований модель не знает

Как это работает

Сходимость вместо идеального шага

- Каждый шаг почти наверняка не идеален
- Конец работы — неподвижная точка
- Оптимизируем скорость, а не красоту шага
- Короче оборот и меньше оборотов
- Управляем циклом: сигнал → правка → сигнал



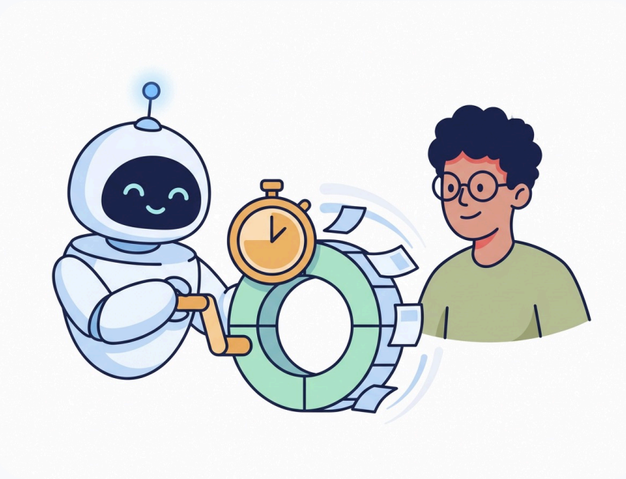
Рычаги скорости

Короче оборот

- Быстрая сборка: секунды, не минуты
- Быстрые тесты: сигнал сразу

Меньше оборотов

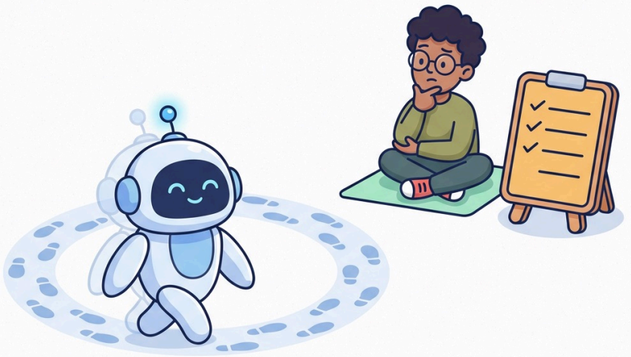
- Типы ловят ошибку до запуска
- Отладчик: состояние, не догадки



Обратная сторона

Когда процесс не сходится

- Нет сигнала: ошибка копится незаметно
- Цель размыта: каждая итерация оптимизирует своё
- Большой шаг: ошибка не локализована
- Мигающий тест: сходимость недостижима в принципе





1

Инструменты

язык · компилятор · среда · отладчик · анализатор · библиотеки

Столп 1 · Инструменты

Что ускоряет агента

Язык

статическая типизация

Сборка

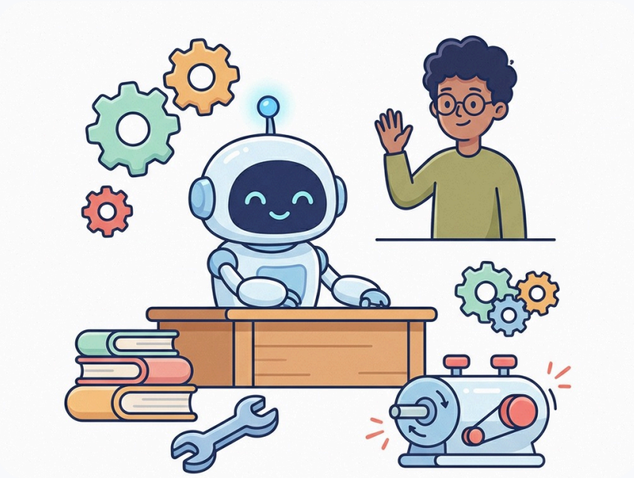
секунды, не минуты

Наблюдаемость

отладчик и анализатор

Библиотеки

UI, сеть, хранение



Инструмент №1

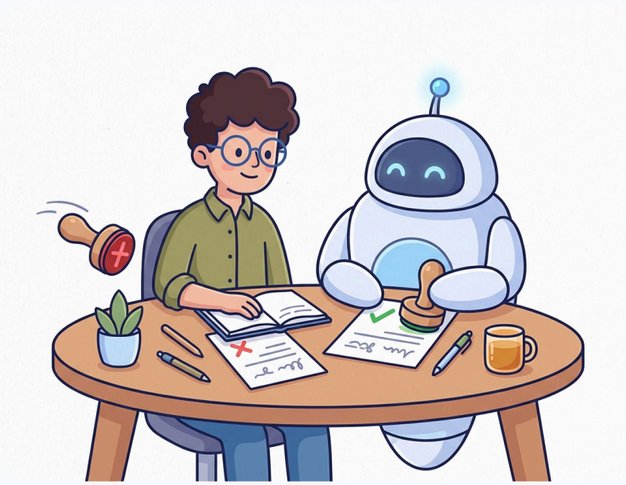
Ошибка видна до запуска

Компилятор проверяет типы до запуска:

ARKTS

```
const n: number = "текст"    // ✗ ошибка типов
const n: number = 42          // ✓ корректно
```

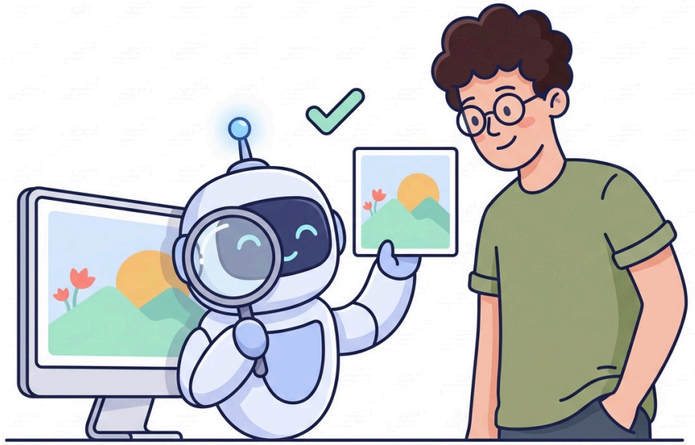
- Агенту не нужно запускать код, чтобы узнать о неверном типе
- Ошибка приходит за секунды и там, где она сделана



Инструмент №2

«Глаза и руки» агента

- Видит отрисованный результат, а не код
- Ходит по дереву интерфейса и кликает
- Сравнивает с ожидаемым состоянием





2

Принципы

стандарты · соглашения · защитные перила

Столп 2 · Принципы

Защитные перила

Никогда: репозиторий

публикация в публичный реестр

перезапись истории

необратимое удаление файлов

Никогда: доступы

запуск чужого кода с правами

правка секретов и ключей

«добавить всё» одной командой

Только по просьбе

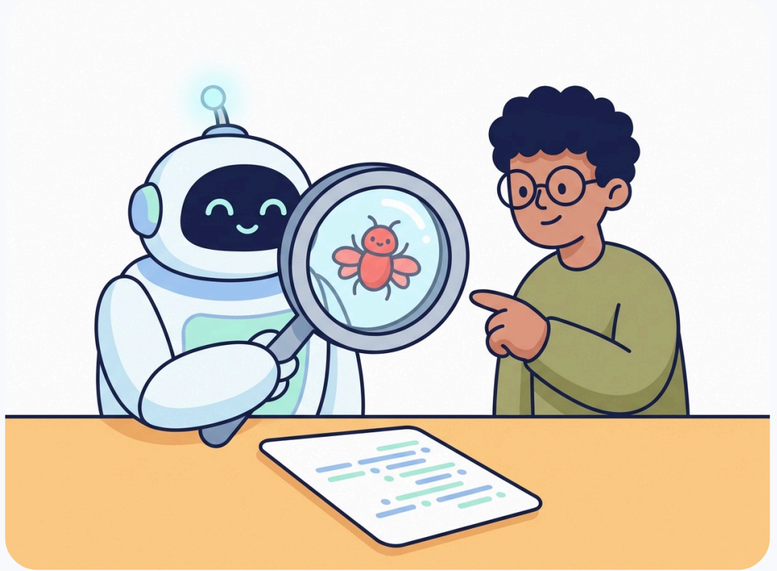
публикация артефактов

массовая очистка репозитория

изменение конфигурации CI

Самопроверка перед сдачей

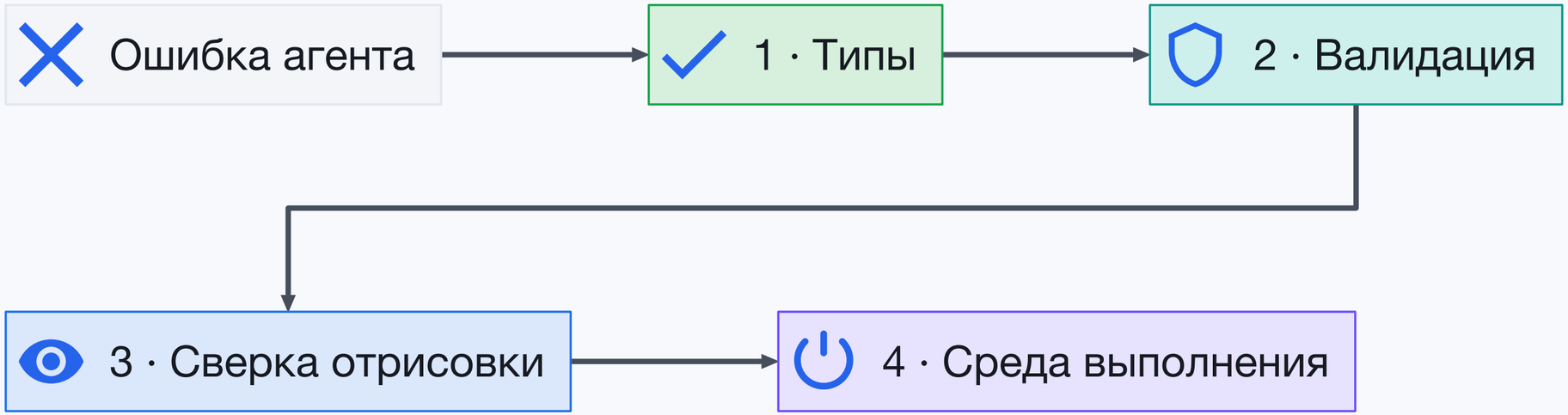
- Чек-лист — по диффу, а не по памяти о правке
- Ищем: неявные преобразования, два способа для «пусто»
- Правки генерённого кода



Итог принципов

Эшелонированная оборона: 4 слоя

Чем раньше слой ловит ошибку — тем дешевле исправление





3

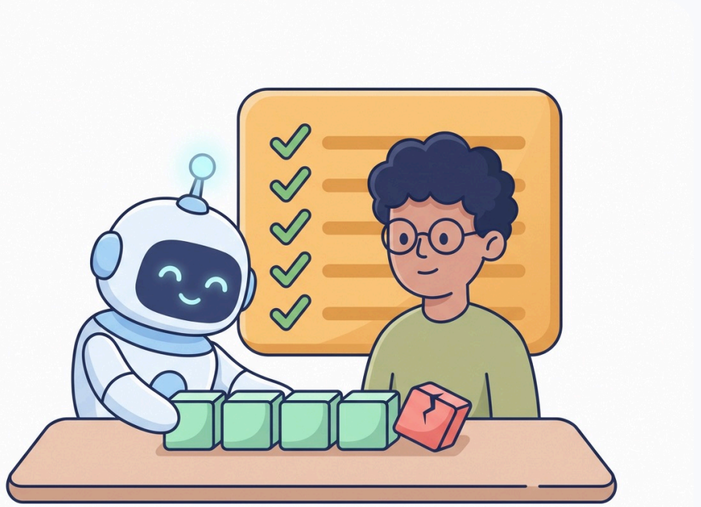
Процессы

документация · ревью · CI · тесты · база знаний

Процессы · Тесты

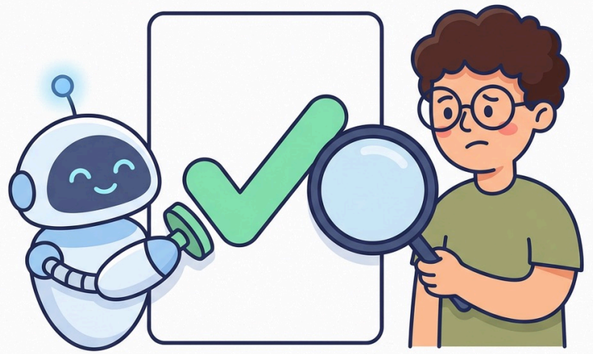
Тесты и CI: предсказуемый агент

- Тесты пишутся вместе с функцией, а не после
- Граничные случаи — там, где живут ошибки
- CI: установка, сборка и тесты на каждом коммите



Зелёный сигнал должен что-то значить

- Упал — а код возврата 0: читать итоговую строку
- Тест без проверок: печать «ок» не доказывает
- Только типичный случай: ошибки живут на краях



Процессы · CI

Красный пайплайн — это задача

1 Упало

лог, шаг, первая ошибка

2 Понять

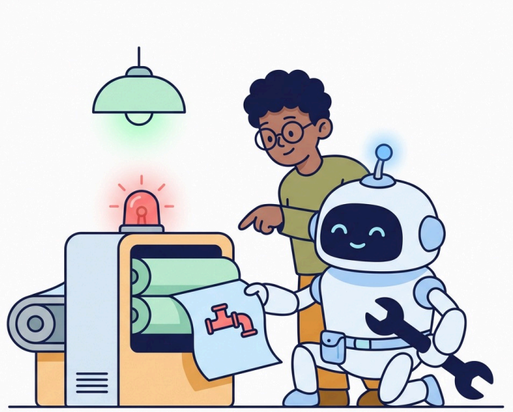
воспроизвести локально

3 Починить

правкой в репозитории

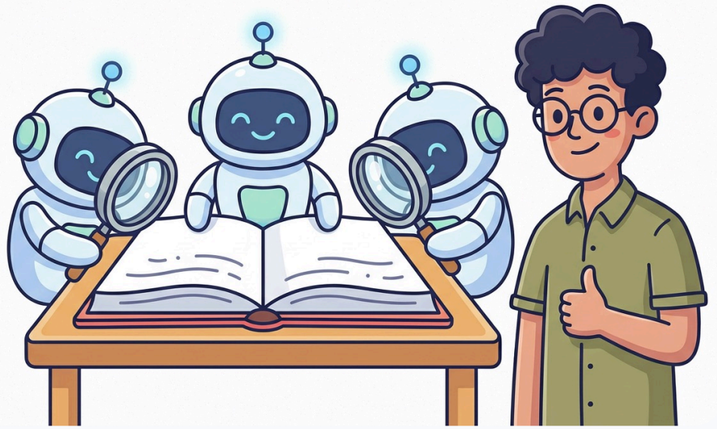
4 Перезапустить

зелёный не случаен?



Ревью: независимые взгляды

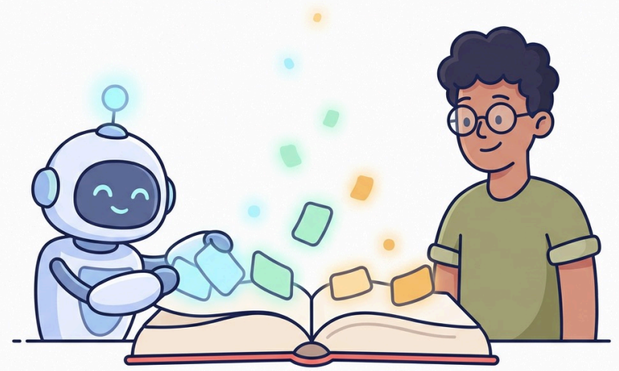
- Четыре роли: архитектура, безопасность, скорость, соглашения
- У каждого свой чек-лист рядом с определением
- Одобряет человек: иначе проверка сертифицирует сама себя



Процессы · Знание

База знаний и память агента

- «Почему» и «как связано» — рядом с кодом
- Заметки живут между сессиями агента
- Обновляется вместе с изменением, а не после



Пример · Как это выглядит

Один конкретный стек как иллюстрация

Что даёт агенту дружелюбный инструментарий

- Компиляция и запуск — секунды, не минуты
- Статические типы ловят ошибку до запуска
- Тесты без экрана и «глаза» на интерфейс
- Внешняя память: правила и заметки рядом с кодом



Пример такого стека: gitcode.com/rri_opensource/arkui3

Что взять с собой

Чек-лист агентной разработки

Что не проверяется машиной — того не существует

Правила

файл, который агент читает до работы

Короткий цикл

компиляция и проверка за секунды

Ранние ошибки

типы и валидация до запуска

«Глаза»

снимок и дерево вместо слепого кода

Тесты с кодом

и запуск на каждом коммите

База знаний

«почему» и «как связано»